Source Language Issues in Run-Time Environments

When a program is written in a high-level language, it must be translated and executed properly at run time.
Different source languages (like C, C++, Java, Python) have different **features and requirements**, which affect how the **run-time environment (RTE)** is designed.
These differences are called **Source Language Issues**.

---

1. Data Type and Storage Requirements

Each source language supports different data types (int, float, array, record, object, etc.).
The compiler must know how much memory each type needs and how it is aligned in memory.

Example:
In C, an `int` might take 4 bytes, while in another language it may take 2 bytes.
This affects **memory allocation** in the runtime environment.

---

2. Scope and Lifetime of Variables

Languages have different rules for **variable scope** (where a variable can be used) and **lifetime** (how long it exists in memory).

- **Static scope:** Used in most languages like C and Java.
- **Dynamic scope:** Used in some older languages like Lisp.

The RTE must manage memory according to these scoping rules.

---

3. Parameter Passing Mechanisms

When functions are called, parameters can be passed in different ways:

- **Call by value** (copy of value)
- **Call by reference** (address of variable)
- **Call by name/value-result**

The runtime system must handle these correctly as per the source language.

---

4. Nested Procedures and Block Structure

Some languages (like Pascal, Ada) allow **nested functions**, while others (like C) do not.
If a language allows nesting, the runtime environment must maintain **access links** or **static chains** to access variables of outer functions.

---

5. Recursion Support

Languages like C, Python, or Java allow **recursive functions**.
This requires **stack-based memory allocation** for activation records of each recursive call.

---

6. Dynamic Data Structures

Languages like C and Java allow **dynamic memory allocation** (`malloc`, `new`).
The RTE must provide mechanisms for:

- Allocating memory from the heap
- Freeing unused memory (garbage collection)

---

7. Exception Handling and Error Control

Some languages have built-in **exception handling** (like `try-catch` in Java).
The runtime system must manage control transfer when an exception occurs.

---

8. Object-Oriented Features

Object-oriented languages like C++ and Java introduce **classes, objects, inheritance, and polymorphism**.
The runtime environment must support:

- Method dispatch tables (for virtual functions)
- Dynamic type checking
- Object memory management

Conclusion

In summary, **source language issues** define how variables, procedures, and data structures are managed during program execution.
The **runtime environment** must be designed according to these language features to ensure correct and efficient program execution.

Storage Organization and Allocation Strategies

When a program is compiled and executed, the **run-time environment** must manage memory efficiently.
It divides memory into different parts to store variables, functions, constants, and data structures.
This process is called **Storage Organization**, and the method used to assign memory to variables is called **Allocation Strategy**.

---

1. Storage Organization

The memory used by a program is divided into **four main areas:**

a) Code Area

- Stores the **compiled machine instructions** (the program code).
- It is usually **read-only** to prevent accidental modification.

b) Static Area

- Stores **global variables**, **static variables**, and **constants** whose size is known at compile time.
- These variables exist for the **entire program execution**.

c) Stack Area

- Used for **function calls**, **local variables**, and **parameters**.
- Each function call creates an **activation record (stack frame)**.
- Memory is allocated and freed automatically when functions are called and return.

d) Heap Area

- Used for **dynamic memory allocation** (e.g., using `malloc` or `new`).
- Memory is managed manually by the programmer or by **garbage collection**.

---

2. Allocation Strategies

There are **three main strategies** for memory allocation during program execution:

a) Static Allocation

- Memory for all data objects is assigned **at compile time**.
- The size and location remain fixed throughout execution.
- No runtime overhead.

- **Used for:** Global and static variables.
  ☑ *Example:* `int x = 10;` (global variable in C)

b) Stack Allocation

- Memory is assigned and freed automatically using a **stack**.
- When a function is called, its local variables and parameters are **pushed onto the stack**.
- When the function returns, they are **popped**.
- **Used for:** Local variables and function calls.
  ☑ *Example:* Function call variables in C.

c) Heap Allocation

- Memory is allocated **at runtime** from the heap using **dynamic allocation**.
- It allows creation of data structures whose size is not known until runtime.
- **Used for:** Linked lists, trees, dynamic arrays.
  ☑ *Example:* `ptr = malloc(sizeof(int));`

---

3. Comparison Table

Strategy	Allocation Time	Lifetime	Example Use
Static	Compile time	Entire program	Global/static variables
Stack	Function call	Function duration	Local variables
Heap	Runtime	Until freed	Dynamic objects

Conclusion

The **storage organization** divides memory into code, static, stack, and heap areas, while **allocation strategies** decide when and how memory is assigned.
Choosing the correct strategy ensures **efficient memory usage** and **faster program execution**.

Parameter Passing

Introduction

When a function or procedure is called, some values or variables are sent from the **calling function** to the **called function**.
The method by which these values are transferred is called **Parameter Passing** or **Parameter Passing Mechanism**.

It defines **how arguments are passed** and **how changes inside the function affect the original variables**.

Types of Parameter Passing Mechanisms

1. Call by Value

- A **copy of the actual value** is passed to the function.
- Changes made inside the function **do not affect** the original variable.
- It is the most common method in languages like **C, Java (for primitive types)**.

☑ *Example:*

```
void add(int x) {
    x = x + 10;
}
int main() {
    int a = 5;
    add(a); // 'a' remains 5
}
```

---

2. Call by Reference

- The **address (reference)** of the variable is passed.
- Any change inside the function **affects the original variable**.
- Used in **C++ (using &)** and **Java (for objects)**.

☑ *Example:*

```
void add(int &x) {
    x = x + 10;
}
int main() {
    int a = 5;
    add(a); // 'a' becomes 15
}
```

---

3. Call by Result

- Memory for the variable is **allocated in the called function**, and the **result is copied back** to the caller after execution.
- The initial value is **not passed**, only the result is returned.
- Used in some older languages like **Ada**.

---

4. Call by Value-Result (Copy-In, Copy-Out)

- A **copy** of the variable is passed (like call by value).
- After the function ends, the **final value** is copied back to the caller (like call by result).
- Combines features of **call by value** and **call by result**.

---

5. Call by Name

- The **expression itself** is passed, not its value.
- The expression is **re-evaluated** every time it is used inside the function.
- Used in **ALGOL** language.

☑ *Example:*
If we call `f(x+1)`, then each time `x+1` is used in `f`, it is re-calculated.

---

6. Call by Need

- Similar to **call by name**, but the expression is **evaluated only once** and the result is **stored for later use**.
- Used in **lazy evaluation languages** like Haskell.

Comparison Table

Method	Value Passed	Can Modify Caller Variable?	Example Language
Call by Value	Copy of value	✗ No	C
Call by Reference	Address	☑ Yes	C++, Java (objects)
Call by Result	No initial value	☑ Yes	Ada
Call by Value-Result	Copy in/out	☑ Yes	Pascal
Call by Name	Expression	☑ Sometimes	ALGOL

Method	Value Passed	Can Modify Caller Variable?	Example Language
Call by Need	Evaluated once	☑ Sometimes	Haskell

Conclusion

Parameter passing mechanisms define how **data moves between functions**.
Choosing the correct method is important for **memory safety**, **program correctness**, and **efficiency** in the **run-time environment**.

Symbol Table Organization and Generation

Introduction

A **Symbol Table** is a **data structure** used by the **compiler** to store information about identifiers (such as variables, functions, classes, objects, etc.) used in a program.
It helps the compiler in **semantic analysis**, **type checking**, **storage allocation**, and **code generation**.

In simple words,

A Symbol Table acts as a *dictionary* for all names (identifiers) in a program.

---

1. Contents of Symbol Table

Each entry in the symbol table contains:

- **Name of identifier** (e.g., variable, function)
- **Type** (integer, float, array, etc.)
- **Scope** (global, local)
- **Memory location / address**
- **Number of parameters** (for functions)
- **Other attributes** (e.g., size, line number)

---

2. Functions of Symbol Table

1. **Insert:** Add a new name when declared.
2. **Lookup:** Search for a name when used.
3. **Modify:** Update information (e.g., type after inference).
4. **Delete:** Remove names (e.g., when leaving a scope).
5. **Display:** Show all stored identifiers (for debugging or optimization).

---

3. Symbol Table Organization

The organization of a symbol table affects how **fast lookup and insertion** can be performed.
Common organizations include:

a) Linear List (Sequential Search)

- All names are stored in a simple list or array.
- Search is done sequentially.
- **Easy to implement but slow** for large programs.
  ☑ *Time complexity:* O(n)

---

b) Hash Table

- Uses a **hash function** to compute an index for each identifier.
- Provides **fast lookup and insertion** on average.
- Collisions are handled by **chaining** or **open addressing**.
  ☑ *Time complexity:* O(1) average case

---

c) Tree (Binary Search Tree / AVL Tree)

- Names are stored in a **sorted order** based on key (identifier name).
- Searching, insertion, and deletion are faster than linear search.
  ☑ *Time complexity:* O(log n)

---

d) Hierarchical (Scoped Symbol Tables)

- Used in **block-structured languages** like C and Pascal.
- Each block or function has its **own symbol table**.
- Linked together to represent **nested scopes** (parent-child relationship).

---

4. Symbol Table Generation (Process)

Symbol table is generated during **compilation**, mainly in the **syntax and semantic analysis** phases:

1. **Lexical Analysis:**
   - Recognizes identifiers and tokens.
   - Creates initial entries in the symbol table.
2. **Syntax Analysis:**
   - Checks structure (declarations, scopes).
   - Updates symbol table with block-level information.
3. **Semantic Analysis:**
   - Adds attributes like data types, scope, and memory location.
4. **Code Generation:**
   - Uses information from the symbol table for address and label resolution.

---

5. Example

```
int x, y;
float area(float r) {
   float pi = 3.14;
```

```
    return pi * r * r;
}
```

Symbol Table:

Name	Type	Scope	Memory	Attributes
x	int	Global	1000	Variable
y	int	Global	1004	Variable
area	function	Global	2000	Returns float
r	float	Local	3000	Parameter
pi	float	Local	3004	Constant

Conclusion

The **Symbol Table** is an essential part of the compiler.
It helps in **efficient identifier management**, **scope resolution**, and **memory allocation**.
A well-organized symbol table improves **compiler performance** and **reduces lookup time**.

Dynamic Storage Allocation

Introduction

During program execution, memory must be managed efficiently to store data, variables, and objects.
When the **exact size and number of data items** are not known at compile time, memory must be allocated **dynamically** at **run time**.
This process is called **Dynamic Storage Allocation**.

It allows programs to request and release memory during execution as needed.

---

1. Need for Dynamic Allocation

- In some cases, **array sizes**, **linked lists**, or **trees** are not known until the program runs.
- Static allocation cannot handle such cases because it fixes memory size at compile time.
- Dynamic allocation provides **flexibility** and **efficient memory usage**.

☑ *Example (C language):*

```
int *p;
p = (int*) malloc(10 * sizeof(int));  // Memory allocated at runtime
```

---

2. Memory Areas Used

Dynamic storage is usually managed in the **Heap area** of memory.
The **Heap Manager** keeps track of:

- **Allocated blocks** (used memory)
- **Free blocks** (available memory)

---

3. Basic Operations

Dynamic allocation supports two basic operations:

a) Allocation

Request memory from the heap.
If available, the system returns a pointer to a free block.

☑ *Example:* `malloc()` or `new`

b) Deallocation

Return the unused memory back to the system so it can be reused.

☑ *Example:* `free()` or `delete`

---

4. Allocation Strategies

There are different **algorithms** to allocate memory from the heap:

a) First Fit

- Allocates the **first free block** large enough to satisfy the request.
- Simple but may leave small unused spaces (fragmentation).

b) Best Fit

- Searches for the **smallest available block** that fits the request.
- Reduces wasted space but is slower due to searching.

c) Worst Fit

- Allocates the **largest available block**.
- Leaves big free spaces for future large allocations.

---

5. Problems in Dynamic Allocation

- **Fragmentation:** Free memory gets divided into small pieces.
  - *External Fragmentation:* Free blocks are scattered.
  - *Internal Fragmentation:* Unused space inside an allocated block.
- **Memory Leaks:** Memory not freed properly after use.
- **Dangling Pointers:** Accessing memory after it's freed.

---

6. Garbage Collection

In some languages (like Java, Python), **garbage collection** automatically frees memory that is no longer used.
This helps prevent **memory leaks**.

---

7. Advantages

- Efficient use of memory
- Flexibility for variable-sized data structures
- Supports recursion and dynamic data structures (linked lists, trees, graphs)

Conclusion

Dynamic Storage Allocation is a key function of the **run-time environment**, enabling flexible and efficient memory management.
It supports complex and data-driven programs where memory requirements change during execution.